EQUIPO 4	
García Cerda Randal Guillermo (2112493 ITS) Sarmiento Hernández Cesar Martín (2119427 ITS) Torres Pasillas Angela Susana (2101844 ITS)	
Nombre del curso: Componentes de Lógica de Negocios	**Nombre del profesor:** Miguel Ángel Gómez Marroquín
Módulo Netacad: 5	**Actividad:** Proyecto integrador (PIA)
Fecha: 06/11/2026	

Bibliografía (Formato APA):

- Evans, E. (2003). Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley.
- Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.
- Nygard, M. (2011). Documenting Architecture Decisions.
- Wiggins, A. (2017). The Twelve-Factor App. https://12factor.net
- FastAPI. Documentación oficial. https://fastapi.tiangolo.com
- Cisco Networking Academy. Componentes de Lógica de Negocios, notas de clase y laboratorios.

Objetivo:

Desarrollar e implementar VetCare 24, una API de lógica de negocio para una clínica veterinaria, aplicando Domain-Driven Design, pruebas automatizadas e integración continua, que cubra citas, expediente clínico, vacunación, hospitalización y emergencias, y que sea trazable desde las necesidades del negocio hasta el código.

1. Resumen ejecutivo

VetCare 24 es una clínica veterinaria que atiende consultas, vacunación, estudios, hospitalización y emergencias. Su operación dependía en parte de expedientes en papel, lo que dificultaba consultar antecedentes, evitar registros duplicados y dar seguimiento oportuno a las vacunas. A partir del levantamiento de requerimientos del Módulo 2 y del diseño de arquitectura del Módulo 3, el equipo construyó la capa de lógica de negocio del sistema con Domain-Driven Design, una arquitectura por capas con puertos y adaptadores, una API REST documentada con OpenAPI y una base de datos PostgreSQL, todo ello empaquetado con Docker Compose y verificado mediante integración continua.

El producto implementado cubre los cinco casos de uso del diseño, de punta a punta, desde la regla de dominio hasta la API y las pruebas: CU-01 Solicitar cita, CU-02 Generar nota clínica (expediente clínico), CU-03 Registrar vacunación, CU-04 Registrar hospitalización y CU-05 Atender una emergencia. Además se implementaron las tareas que el BPMN dejaba sin código: recetas y órdenes de estudio (US13, US14), cobro de citas con entrega de documentación (US10) y el reporte de operación para el responsable (NB-21). Quedan fuera de alcance la factura fiscal, el envío real de recordatorios y la autenticación; el informe los declara como limitaciones.

Tabla 1. Resumen de resultados.

Indicador	Resultado
Casos de uso implementados	5 de 5 (CU-01 a CU-05)
Historias de usuario analizadas	20, todas con criterios Gherkin; 16 implementadas, 3 parciales y 1 fuera del alcance
Pruebas automatizadas	696 aprobadas en la integración continua (SQLite y PostgreSQL 16), incluidas las del paquete del profesor y las de los cinco casos de uso
Cobertura de código	99.14 % con medición de ramas; umbral exigido: 80 %
Verificación contra el paquete del profesor	120 casos de prueba y 20 transiciones coinciden con un oráculo independiente
Escenarios de emergencias por HTTP	20 de 20 coinciden con el resultado derivado de los requerimientos
Decisiones de arquitectura (ADR)	10 (ADR-09 revisado para reflejar los cinco casos de uso)
Integración continua	GitHub Actions: lint, pruebas con PostgreSQL, evidencia y construcción de la imagen; rama `main` protegida

2. Análisis del negocio (BRD)

2.1 Contexto y problemas

El BRD completo se conserva en `docs/BRD.md` del repositorio. A continuación se resumen los elementos que gobiernan el diseño. La clínica opera con cinco actores: el propietario, la mascota, Recepción, el veterinario y el personal clínico, además de un responsable de la clínica que supervisa la operación. El proceso comienza con el registro del propietario y de la mascota y la programación de una cita; después vienen la recepción y la atención, y en los casos de vacunación, hospitalización o emergencia se requieren procesos adicionales de seguimiento y registro.

Tabla 2. Problemas identificados en el BRD.

ID	Problema	Descripción
P-01	Expedientes en papel	Parte de la información clínica se mantiene en papel, lo que dificulta su consulta y seguimiento.
P-02	Expedientes duplicados	Existe el riesgo de registrar una misma mascota más de una vez, generando información duplicada.
P-03	Consulta lenta del historial	Es difícil consultar rápidamente los antecedentes clínicos de una mascota cuando se necesita información previa.
P-04	Recordatorios de vacunas deficientes	Los recordatorios actuales no siempre permiten dar un seguimiento adecuado a las vacunas próximas o a los refuerzos.
P-05	Información clínica dispersa	La información necesaria para la atención puede encontrarse en diferentes registros, dificultando su consulta.
P-06	Seguimiento de hospitalización	Los pacientes hospitalizados requieren seguimiento continuo y registro de su evolución.
P-07	Registro de emergencias	Las emergencias pueden presentarse fuera del flujo normal de citas, por lo que se requiere registrar la atención sin una cita previa.

2.2 Necesidades del negocio y estado de implementación

El BRD define 21 necesidades de negocio. La columna final indica si el código entregado las satisface, de modo que el lector pueda distinguir con claridad lo implementado de lo que permanece como diseño.

Tabla 3. Necesidades del negocio (NB) y su estado.

ID	Necesidad del negocio	Estado de implementación
NB-01	Mantener un registro de los propietarios de las mascotas.	Implementada
NB-02	Permitir asociar varias mascotas a un mismo propietario.	Implementada
NB-03	Mantener un registro individual para cada mascota.	Implementada
NB-04	Registrar los datos relevantes de cada mascota, como especie, raza, sexo, fecha de nacimiento y peso.	Implementada
NB-05	Mantener registrada la información relevante para la atención, como alergias y cambios de peso.	Parcial (alergias y peso como atributos de la mascota)
NB-06	Gestionar las citas de acuerdo con el servicio veterinario solicitado.	Implementada
NB-07	Considerar la duración requerida por los diferentes tipos de servicio al gestionar las citas.	Diferida (sin horarios ni duraciones en los datos)
NB-08	Permitir modificar o cancelar las citas cuando el propietario lo solicite.	Implementada
NB-09	Registrar la llegada de la mascota y su motivo de consulta.	Parcial (confirmación de la cita; peso y motivo de consulta no se capturan como flujo propio)
NB-10	Mantener un expediente clínico actualizado para cada mascota.	Implementada (CU-02)
NB-11	Facilitar al personal autorizado la consulta de los antecedentes clínicos de una mascota.	Implementada (CU-02)
NB-12	Registrar las notas clínicas generadas durante la atención veterinaria.	Implementada (CU-02)
NB-13	Registrar los medicamentos, estudios, vacunas o indicaciones de seguimiento determinadas por el veterinario.	Implementada (CU-02: nota con indicaciones, recetas y órdenes de estudio)
NB-14	Mantener un historial de las vacunas aplicadas a cada mascota.	Implementada
NB-15	Gestionar la frecuencia o esquema de aplicación de las vacunas y determinar las próximas aplicaciones.	Implementada
NB-16	Identificar las vacunas que requieren refuerzo y darles el seguimiento correspondiente.	Implementada
NB-17	Mejorar los recordatorios de vacunas próximas para los propietarios.	Implementada (consulta de próximas; sin envío)

ID	Necesidad del negocio	Estado de implementación
NB-18	Registrar y dar seguimiento a las mascotas que se encuentren hospitalizadas.	Implementada (CU-04)
NB-19	Permitir registrar y gestionar emergencias que ocurran sin una cita previamente programada.	Implementada
NB-20	Controlar los permisos de los usuarios de acuerdo con su rol.	Implementada
NB-21	Generar reportes que permitan supervisar la operación de la clínica.	Implementada (`GET /reportes/operacion`)

2.3 Restricciones y criterios de éxito

Las catorce restricciones (R-01 a R-14) y los diez criterios de éxito (CR-1 a CR-10) completos están en el BRD. Las restricciones que más influyeron en el diseño son: Recepción administra citas y datos administrativos pero no modifica notas clínicas (R-01, R-03); las emergencias pueden presentarse sin cita (R-06); la información de vacunas conserva su historial y su esquema debe estar definido y validado antes de calcular próximas aplicaciones (R-11, R-12, R-13); y los permisos se relacionan con las responsabilidades de cada rol (R-09, R-14).

2.4 Historias de usuario y criterios de aceptación

Se redactaron 20 historias de usuario con prioridad y escenarios en Gherkin (`docs/historias_de_usuario.md`). La siguiente tabla las relaciona con su estado de implementación; el detalle por componente y prueba está en la matriz de trazabilidad.

Tabla 4. Historias de usuario y estado de implementación.

US	Historia	Estado	Dónde
US01	Registrar mascota	Implementada	`POST /mascotas`
US02	Registrar múltiples mascotas	Implementada	`POST /mascotas` (varias por propietario)
US03	Consultar expediente (propietario)	Implementada	`GET /mascotas/{id}/expediente` (rol responsable), `GET /mascotas/{id}/vacunaciones`
US04	Solicitar cita	Implementada	`POST /citas` y ciclo completo
US05	Recordatorios	Parcial (solo vacunas, sin envío)	`GET /vacunas/proximas`
US06	Recepción registra datos	Implementada (alta)	`POST /mascotas`

US	Historia	Estado	Dónde
	de mascota		
US07	Consultar citas programadas	Implementada	`GET /citas`
US08	Registrar peso al llegar	Parcial (peso como dato de la mascota)	`Mascota.peso_kg`
US09	Registrar motivo de consulta	Fuera del alcance implementado (el motivo existe solo en emergencias)	—
US10	Registrar cobro	Implementada	`POST /citas/{id}/cobro`
US11	Veterinario consulta expediente	Implementada	`GET /mascotas/{id}/expediente`
US12	Registrar nota clínica	Implementada	`POST /mascotas/{id}/notas-clinicas`
US13	Registrar receta	Implementada	`POST /mascotas/{id}/recetas`
US14	Órdenes de estudios	Implementada	`POST /mascotas/{id}/ordenes-estudio`
US15	Registrar vacunas	Implementada	`POST /mascotas/{id}/vacunaciones`
US16	Consultar alergias	Parcial (atributo de la mascota)	`Mascota.alergias`
US17	Ingreso a hospitalización	Implementada	`POST /hospitalizaciones`
US18	Hoja de monitoreo	Implementada	`POST /hospitalizaciones/{id}/monitoreo`
US19	Evolución del hospitalizado	Implementada (las lecturas de la hoja registran la evolución)	`POST /hospitalizaciones/{id}/monitoreo`, `GET /hospitalizaciones/{id}`
US20	Atender una emergencia	Implementada	`POST /emergencias`

Como ejemplo, el criterio de aceptación de la emergencia (US20) se expresa así:

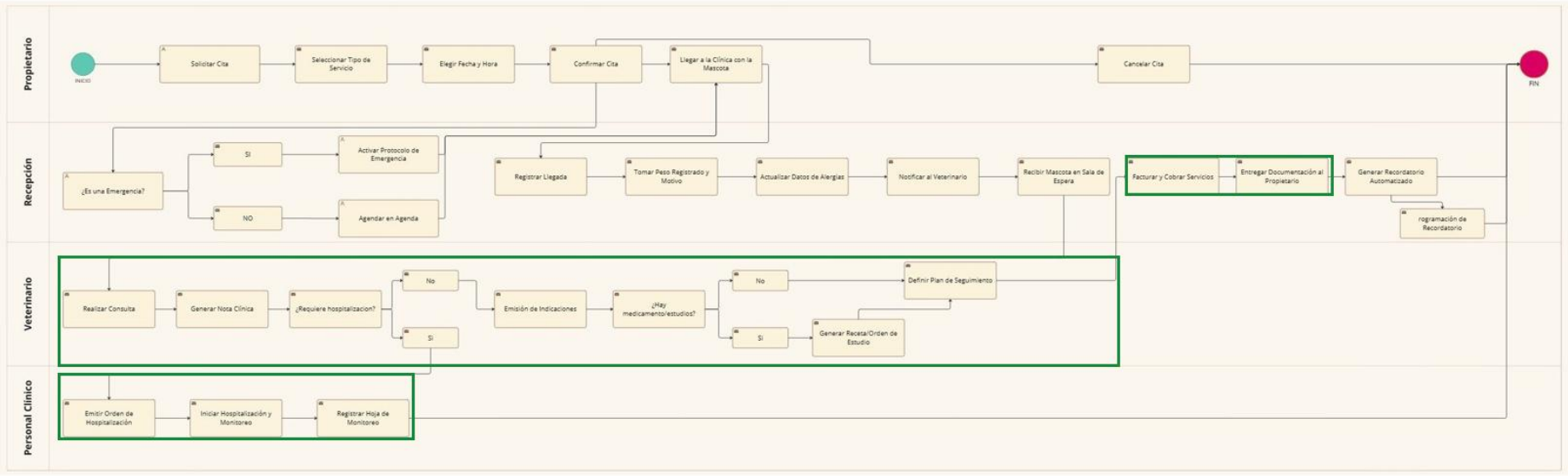
Característica: Atención de emergencia — Escenario: Identificar una emergencia. Dado que la mascota ha llegado a la clínica, cuando Recepción determina que es una emergencia, entonces debe activarse el protocolo de emergencia y debe notificarse al veterinario.

2.5 Glosario del dominio

El glosario de 20 términos (`docs/glosario.md`) fija el lenguaje ubicuo del equipo. Términos como Cita Médica, Emergencia, Esquema de Vacunación y Refuerzo de Vacuna se reflejan directamente en los nombres de los agregados y servicios del código; la última columna del glosario indica dónde vive cada término en el código.

2.6 Proceso de negocio (BPMN)

El proceso de atención se modeló con cuatro carriles: Propietario, Recepción, Veterinario y Personal clínico. La figura siguiente muestra el modelo: en verde, las tareas implementadas en el código, incluidos los carriles de Veterinario y Personal clínico (consulta, nota clínica, indicaciones, receta u orden de estudio y hospitalización con hoja de monitoreo) y, en Recepción, el cobro y la entrega de documentación.



Verde: tareas implementadas en el código. Sin marca: también implementadas (citas, emergencias, vacunas).

Figura 1. BPMN del proceso de atención. En verde, las tareas implementadas en el código.

3. Alcance implementado

El diseño de la Unidad 3 modela cinco casos de uso. Los cinco se implementaron con el mismo nivel de profundidad: reglas de dominio, máquina de estados, eventos, persistencia, API, pruebas, CI y despliegue. En la primera versión de la entrega, CU-02 y CU-04 se habían dejado como diseño (ADR-09); al revisar el informe se concluyó que dejar dos casos de uso sin código volvía difícil justificar el alcance, por lo que se implementaron y el ADR-09 se revisó en consecuencia.

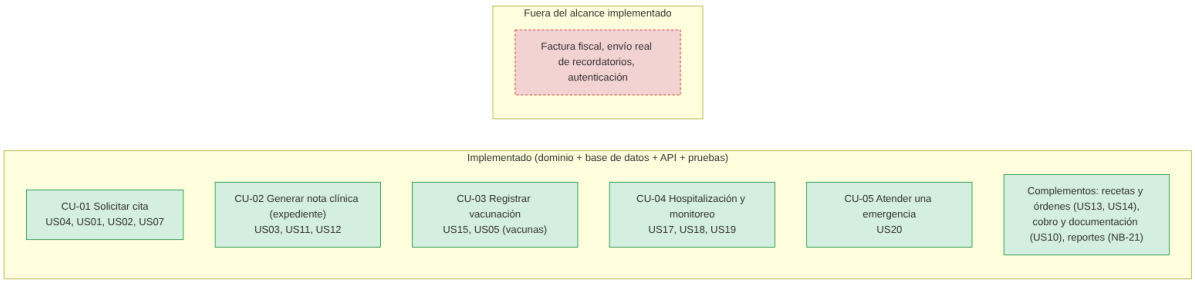


Figura 2. Casos de uso implementados y elementos fuera del alcance.

Tabla 5. Alcance por caso de uso.

Caso de uso	Historias	Estado
CU-01 Solicitar cita	US04, US01, US02, US07	Implementado
CU-02 Generar nota clínica (expediente)	US03, US11, US12	Implementado
CU-03 Registrar vacunación	US15, US05 (vacunas)	Implementado
CU-04 Registrar hospitalización	US17–US19	Implementado
CU-05 Atender una emergencia	US20	Implementado

CU-02 incorpora el agregado inmutable `NotaClinica` (el expediente de una mascota es la lista de sus notas) y el servicio de dominio `RegistroClinico`; una nota no se modifica ni se borra, con lo que se cumple R-03, y puede ligarse a una cita realizada de la misma mascota. CU-04 incorpora el agregado `Hospitalizacion` con su hoja de monitoreo (`LecturaDeMonitoreo`), una máquina de estados INGRESADA → ALTA y la regla de una sola hospitalización ingresada por mascota. Ambos emiten los eventos `NotaClinicaRegistrada`, `HospitalizacionIniciada` y `HospitalizacionFinalizada`. Las decisiones concretas están en la sección O de `decisiones.md` (O-01 a O-09). Para cerrar todas las tareas del BPMN se agregaron además `Receta` y `OrdenDeEstudio` (documentos inmutables del veterinario, ligables a una cita realizada), el cobro

de citas (`Cobro`: solo Recepción, una sola vez por cita realizada y por su total vigente), la entrega de documentación (comprobante, notas, recetas y órdenes de la cita, solo si ya fue cobrada) y el reporte de operación del responsable (`GET /reportes/operacion`). Estas decisiones son O-10 a O-14. Siguen fuera de alcance la factura fiscal (A-02), el envío real de recordatorios y la autenticación.

4. Arquitectura

4.1 Visión por capas

El sistema se organiza en cuatro capas con dependencias que apuntan hacia el dominio. La capa `domain/` contiene las reglas de negocio puras y no conoce FastAPI ni la base de datos. La capa `application/` orquesta los casos de uso, verifica los permisos por rol y publica los eventos. La capa `infrastructure/` implementa los puertos con PostgreSQL o SQLite, el importador de CSV y el notificador. La capa `api/` expone los casos de uso como una API REST documentada con OpenAPI.

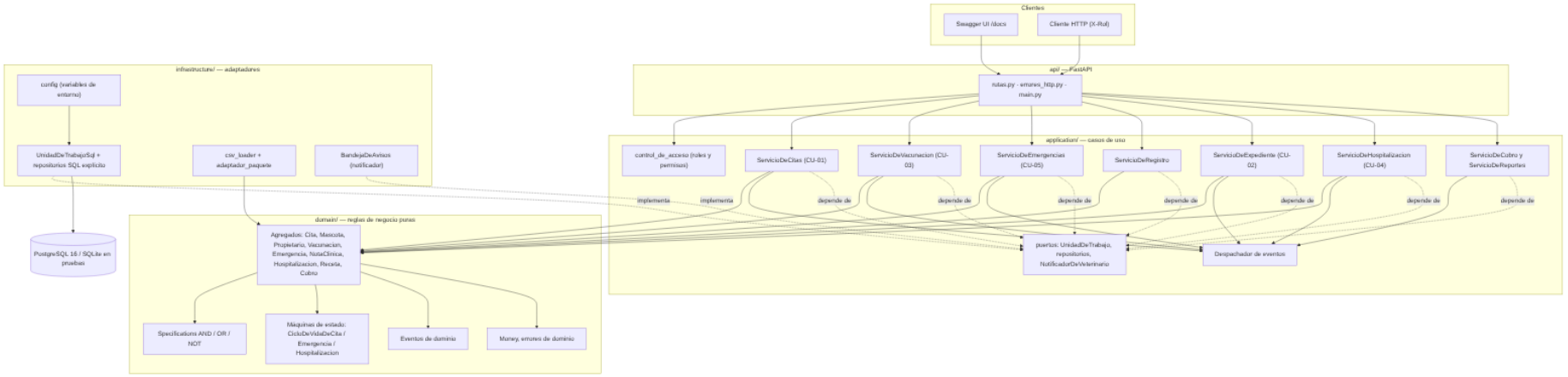


Figura 3. Arquitectura por capas y dependencias.

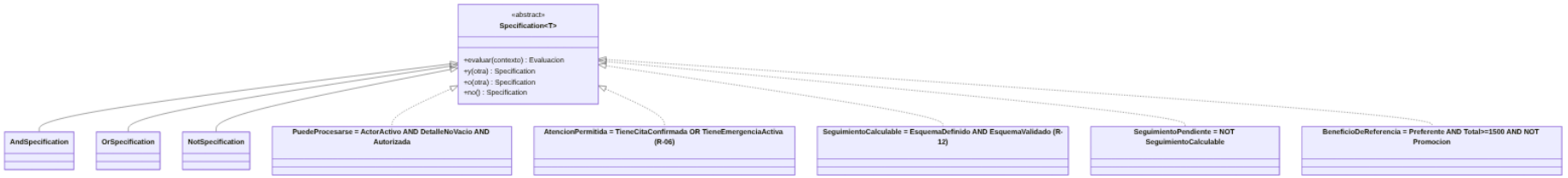


Figura 4. Specifications compuestas del dominio (AND, OR, NOT).

4.2 Modelo de dominio y diagrama de clases

El modelo de dominio sigue Domain-Driven Design: los agregados `Propietario`, `Mascota`, `Cita`, `Vacunacion`, `Emergencia`, `NotaClinica` y `Hospitalizacion` se relacionan por identidad (ADR-02) y cada operación modifica un solo agregado dentro de una unidad de trabajo. Los diagramas de las figuras siguientes provienen del diseño de la Unidad 3; los cinco casos de uso que modelan están implementados, y los nombres del código son equivalentes en español.

Mapa de dominio (diseno) - los cinco casos de uso estan implementados

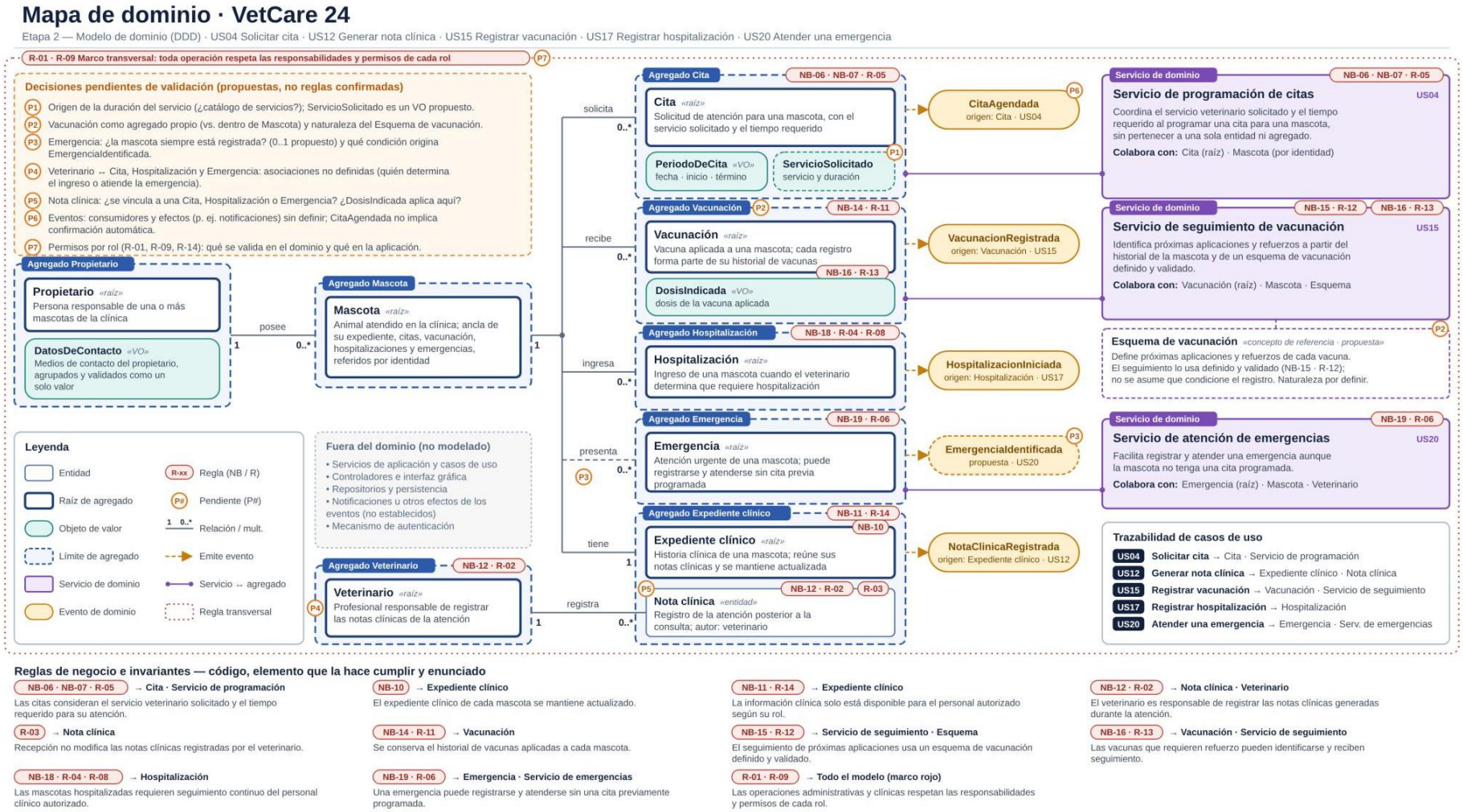


Figura 5. Mapa de dominio (diseño de la Unidad 3).

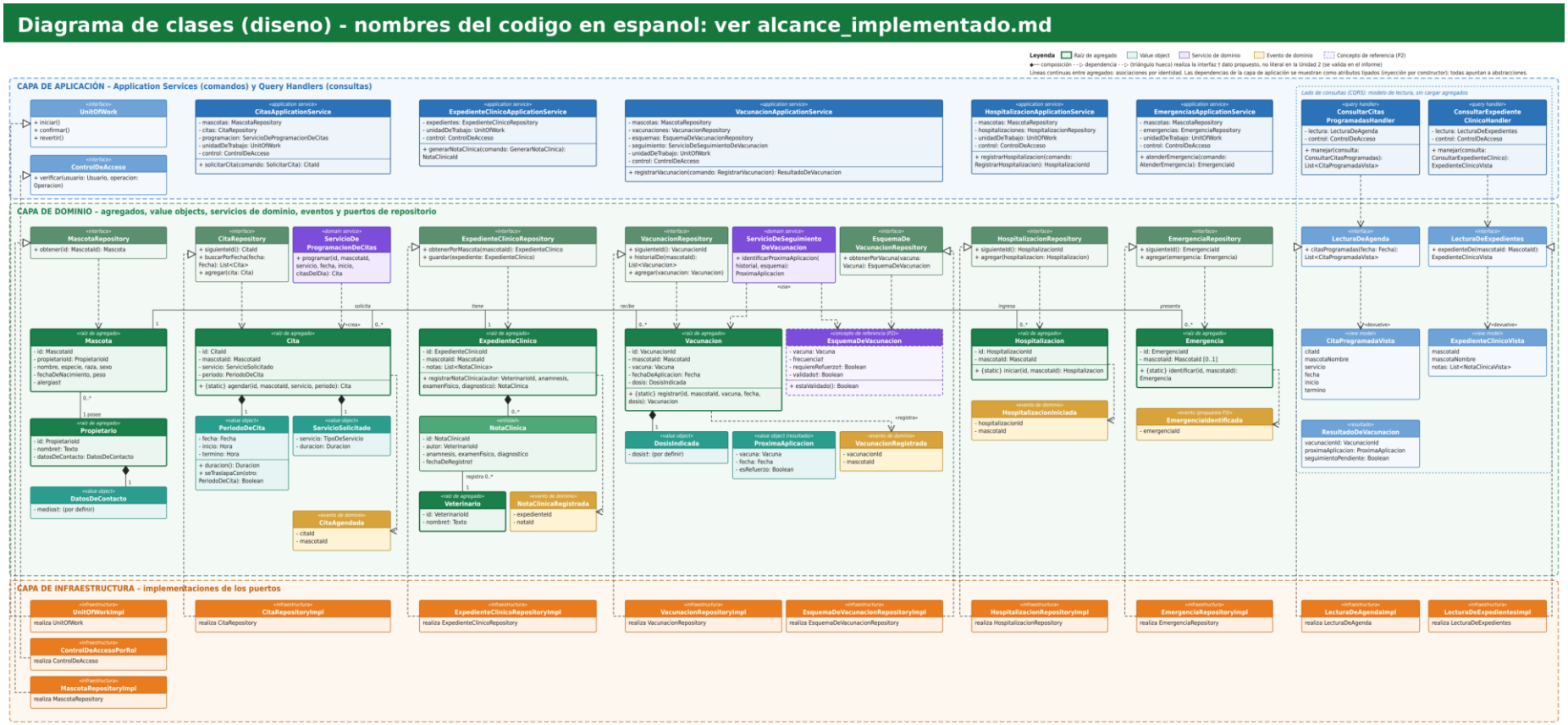


Figura 6. Diagrama de clases UML (diseño de la Unidad 3). Los nombres del código son equivalentes en español.

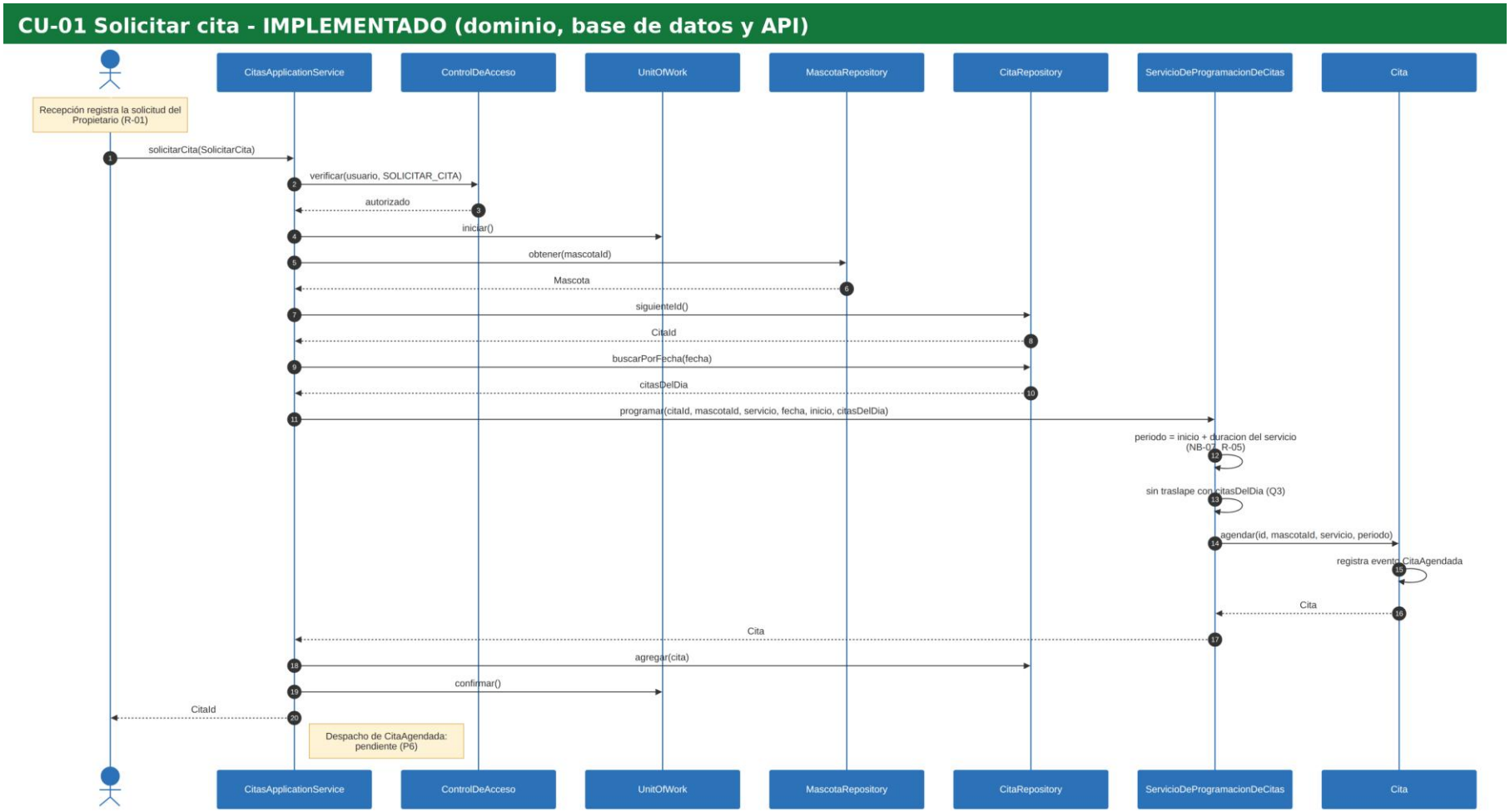


Figura 7. Secuencia de CU-01 Solicitar cita (implementado).

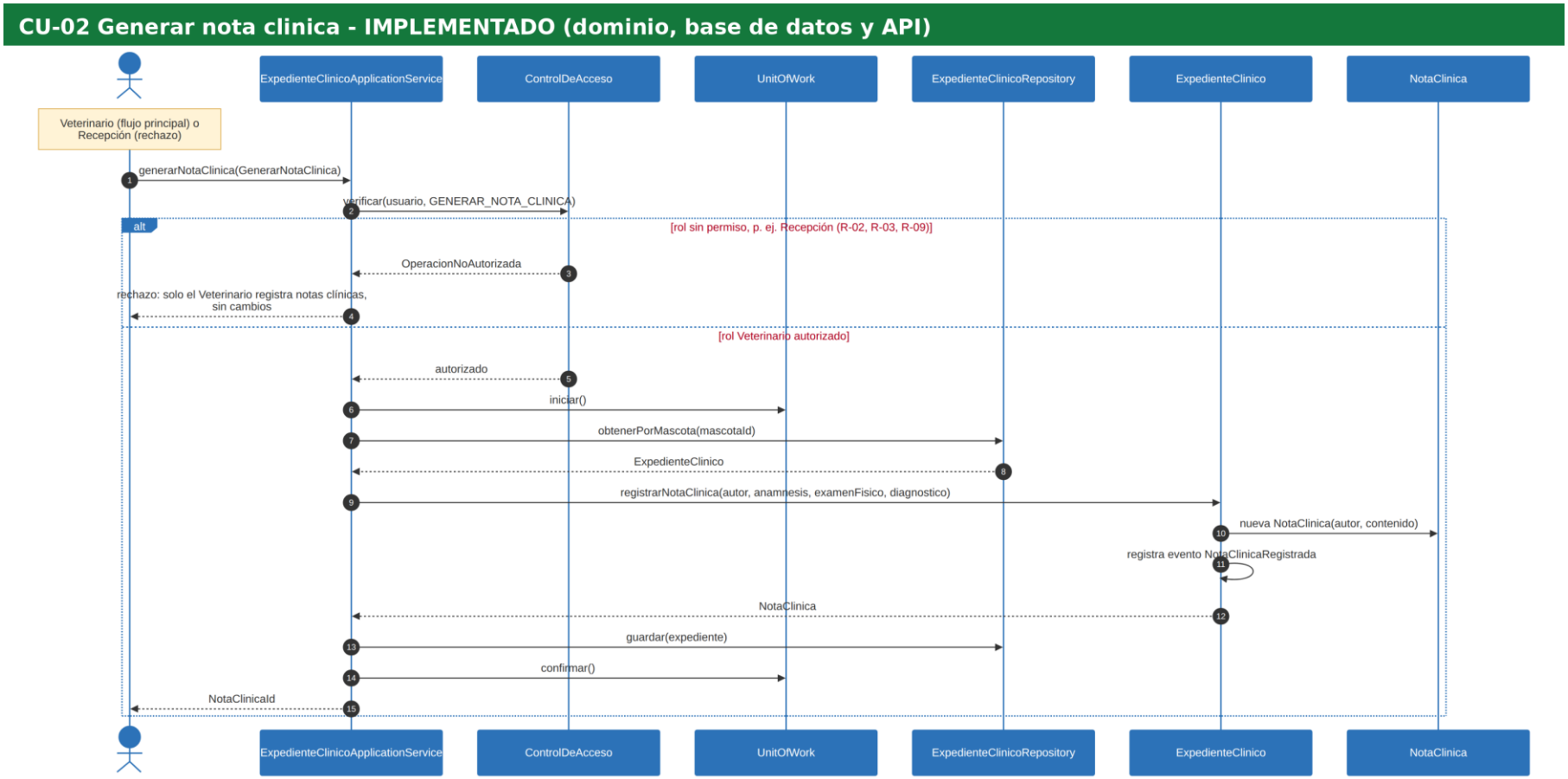


Figura 8. Secuencia de CU-02 Generar nota clínica (implementado).

4.3 Máquinas de estado

La cita sigue el ciclo SOLICITADA → CONFIRMADA → REALIZADA, con cancelación posible desde SOLICITADA o CONFIRMADA; REALIZADA y CANCELADA son estados terminales. La capacidad del servicio se verifica y se consume al confirmar, y cancelar la libera (ADR-07). La tabla `TRANSICIONES` es la única fuente de verdad, y cualquier transición fuera de ella responde 409 `TRANSICION_INVALIDA`. La emergencia tiene un ciclo más corto: IDENTIFICADA → EN_ATENCION, con el cierre fuera de alcance. La hospitalización sigue INGRESADA → ALTA: mientras está ingresada admite lecturas de la hoja de monitoreo, y el alta exige un resumen y cierra la hospitalización (409 ante cualquier cambio posterior).

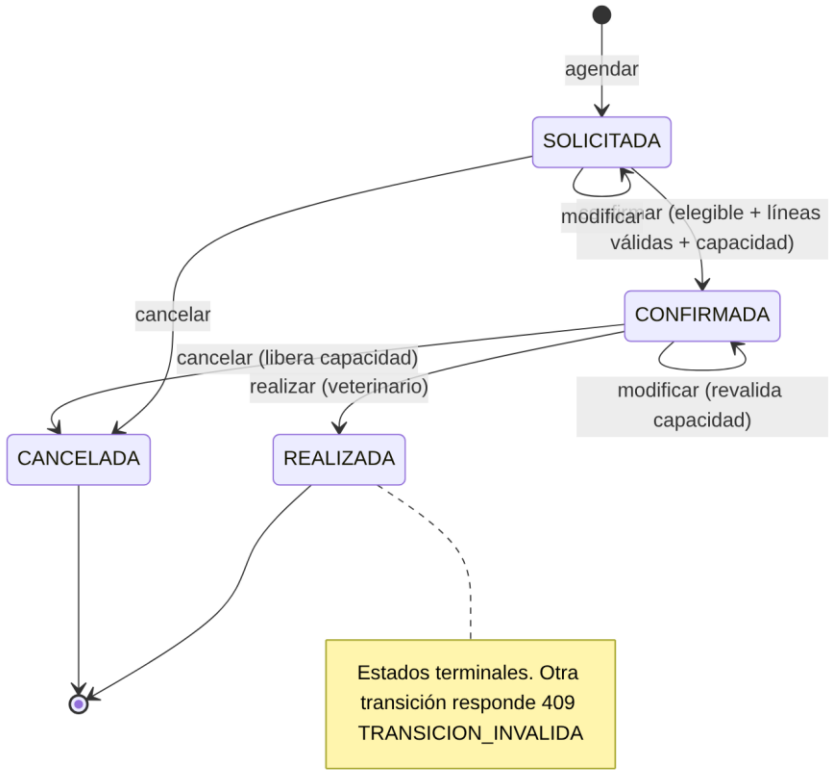


Figura 9. Máquina de estados de la Cita.

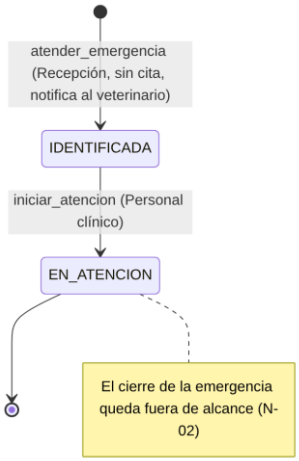


Figura 10. Máquina de estados de la Emergencia.

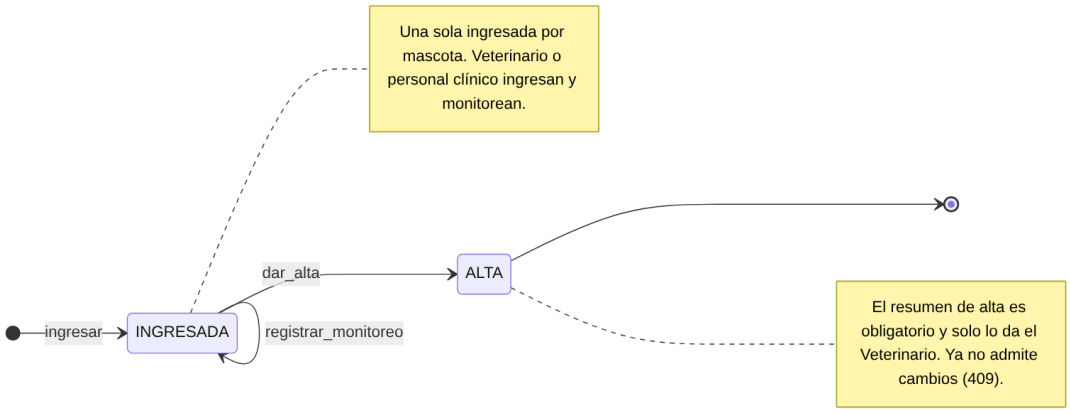


Figura 11. Máquina de estados de la Hospitalización.

4.4 Secuencia de CU-05 Atender una emergencia

Recepción registra la emergencia sin consultar ninguna cita (R-06). El servicio verifica el permiso antes de abrir la transacción, crea la emergencia con el ciclo de vida, guarda la emergencia y sus eventos, confirma y, solo después del commit, publica los eventos y notifica al veterinario. Si el aviso falla, el registro no se deshace y la respuesta indica `pendiente_de_notificar`.

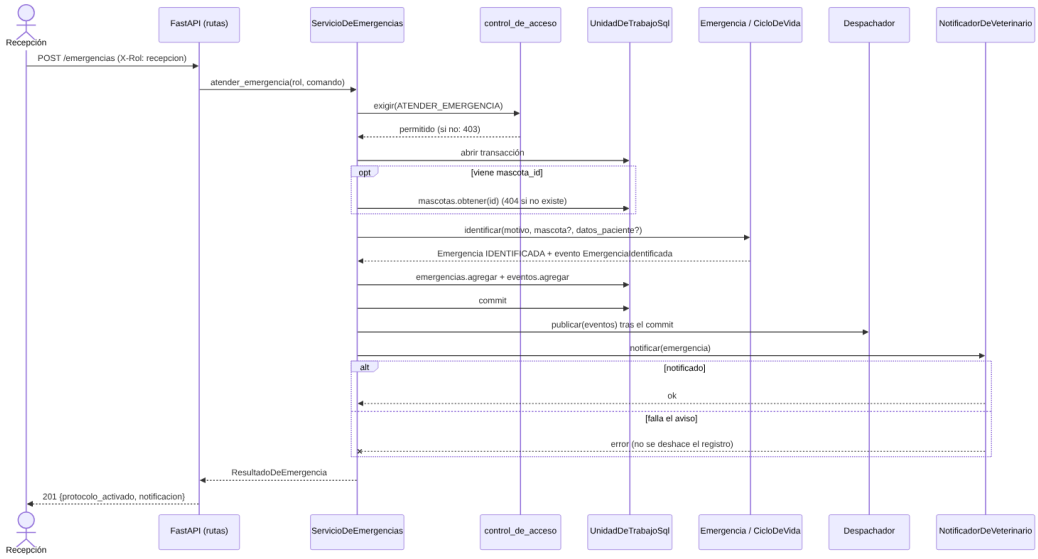


Figura 12. Secuencia de CU-05 Atender una emergencia.

4.4.1 Secuencia de CU-04 Hospitalización

El veterinario o el personal clínico ingresan a la mascota; el servicio verifica el permiso, consulta las hospitalizaciones previas de la mascota y, si ninguna está ingresada, crea la hospitalización con su evento. Mientras está ingresada se agregan lecturas a la hoja de monitoreo y, al final, el veterinario da el alta con un resumen. Los eventos se guardan en la misma transacción y se publican después del commit.

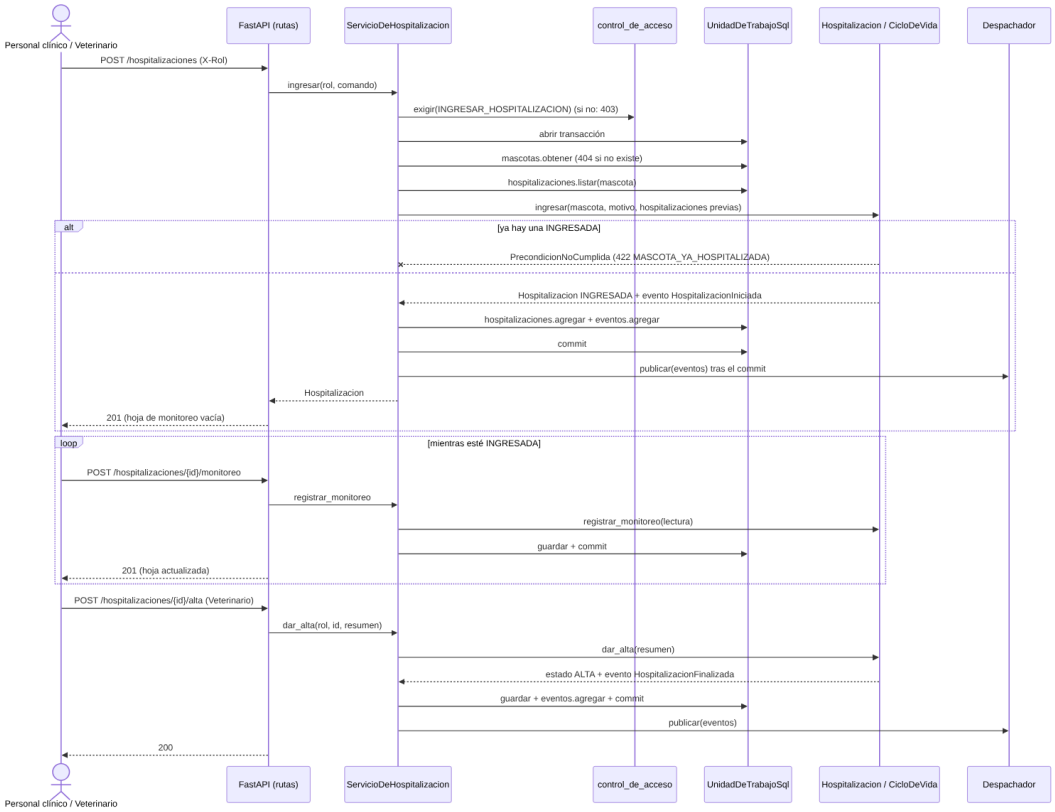


Figura 13. Secuencia de CU-04: ingreso, monitoreo y alta.

4.5 Modelo de datos

La persistencia usa SQL explícito sobre PostgreSQL 16 (SQLite en pruebas rápidas), con restricciones `CHECK` y llaves foráneas. La mascota de una emergencia es opcional (P3), porque la mascota puede no estar registrada. Las notas clínicas, las hospitalizaciones y las lecturas de monitoreo tienen sus propias tablas con restricciones `CHECK` (por ejemplo, el estado ALTA exige fecha y resumen de alta).

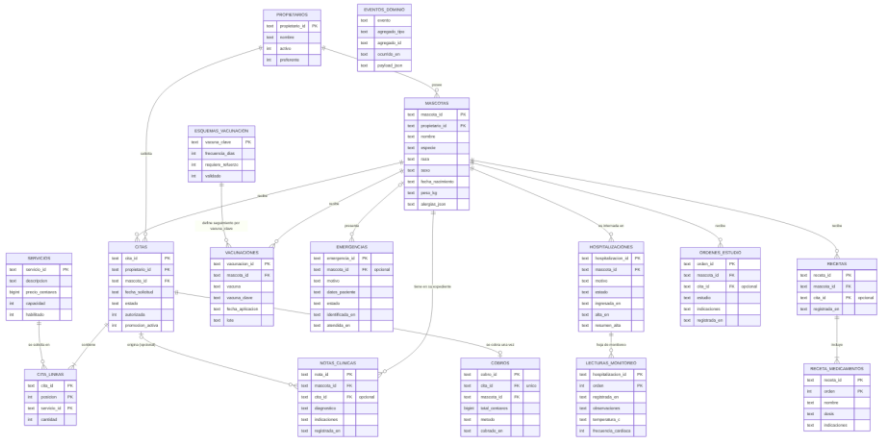


Figura 14. Modelo entidad-relación.

4.6 Patrones de diseño y SOLID

Tabla 6. Patrones aplicados.

Patrón	Dónde y para qué
Domain Model, Aggregate Root	`domain/`: cada regla vive en el objeto que la protege (ADR-01, ADR-02)
Value Object	`Money` (Decimal, 2 decimales, ROUND_HALF_UP) y `ServicioSolicitado`
Specification	Reglas componibles con AND, OR y NOT que explican sus motivos de falla: `PuedeProcesarse`, `AtencionPermitida`, `SeguimientoCalculable`
State (tabla de transiciones)	`ciclo_de_vida.py`: única fuente de verdad de los estados
Repository y Unit of Work	Puertos en `application/puertos.py`; una transacción por caso de uso en `UnidadDeTrabajoSql`
Domain Events y Observer	Eventos de Cita, Vacunación, Emergencia, Nota clínica y Hospitalización, guardados y publicados tras el commit por el `Despachador`
Ports and Adapters, Adapter	El dominio no depende de infraestructura; `adaptador_paquete` aísla el formato de los CSV y `BandejaDeAvisos` implementa el notificador
Factory y Dependency Injection	`CicloDeVidaDeCita.agendar` y `CicloDeVidaDeEmergencia.identificar` crean objetos válidos; reloj, generador de ids y fábrica de unidades de trabajo son inyectables

Los principios SOLID se aplican así: responsabilidad única en módulos como `reglas_cita`, `ciclo_de_vida`, `disponibilidad` y `control_de_acceso`; abierto y cerrado porque las reglas nuevas se componen sin modificar las existentes; sustitución de Liskov porque los repositorios SQL se

comportan igual sobre SQLite y PostgreSQL y el notificador se sustituye en las pruebas; segregación de interfaces con puertos pequeños por agregado; e inversión de dependencias porque los servicios dependen de `puertos.py` y la implementación se inyecta en `main.py`.

4.7 Decisiones de arquitectura (ADR)

Se registraron diez ADR; cada uno cuenta con contexto, alternativas, decisión, consecuencias y trazabilidad en `docs/architecture/adr/`.

Tabla 7. Registro de decisiones de arquitectura.

ADR	Decisión	Estado
ADR-01	Selección de Domain Model como patrón de lógica de negocio	Aceptada
ADR-02	Referencias por identidad y una transacción por caso de uso	Aceptada
ADR-03	CQRS básico: comandos en Application Services y consultas con puertos de lectura	Aceptada con alcance reducido
ADR-04	Dónde se verifican los permisos por rol	Aceptada
ADR-05	Esquema de vacunación como concepto de referencia	Aceptada
ADR-06	Stack tecnológico	Aceptada
ADR-07	Máquina de estados de Cita y momento de consumo de capacidad	Aceptada
ADR-08	Adaptador del paquete de pruebas del profesor	Aceptada
ADR-09	Alcance implementado: los cinco casos de uso	Aceptada
ADR-10	Configuración por variables de entorno y despliegue en contenedor	Aceptada

4.8 API REST y seguridad por rol

La API expone 42 operaciones y su contrato OpenAPI se publica en `/openapi.json` y en Swagger UI (`/docs`). Cada solicitud indica el rol en el encabezado `X-Rol` (`recepcion`, `veterinario`, `personal_clinico`, `responsable`); la verificación de permisos es el primer paso de cada caso de uso (ADR-04). El encabezado modela los permisos por rol pero no constituye autenticación real, que queda fuera del alcance. Los errores se devuelven siempre con el mismo formato y el cuerpo trae solo el mensaje en español, pensado para quien usa el sistema; el código estable y los motivos para programas viajan en los encabezados `X-Codigo-Error` y `X-Motivos` (O-15): 400 solicitud mal formada o rol desconocido, 401 falta de rol, 403 sin permiso, 404 recurso inexistente, 409 transición inválida o capacidad insuficiente y 422 regla de negocio no cumplida.

Tabla 8. Operaciones de la API.

Método	Ruta	Resumen
GET	`/health`	Estado del servicio
GET	`/servicios`	Listar el catálogo de servicios
POST	`/propietarios`	Registrar un propietario
GET	`/propietarios/{propietario_id}`	Consultar un propietario
POST	`/mascotas`	Registrar una mascota
GET	`/mascotas/{mascota_id}`	Consultar una mascota
POST	`/citas`	Solicitar una cita
GET	`/citas`	Listar citas (filtros opcionales)
GET	`/citas/{cita_id}`	Consultar una cita
PUT	`/citas/{cita_id}/servicios`	Modificar los servicios de una cita
POST	`/citas/{cita_id}/confirmar`	Confirmar una cita (SOLICITADA → CONFIRMADA)
POST	`/citas/{cita_id}/realizar`	Realizar una cita (CONFIRMADA → REALIZADA)
POST	`/citas/{cita_id}/cancelar`	Cancelar una cita
GET	`/citas/{cita_id}/evaluaciones/{regla}`	Evaluar una regla de elegibilidad sobre una cita
GET	`/eventos`	Listar los eventos de dominio guardados
POST	`/mascotas/{mascota_id}/vacunaciones`	Registrar la aplicación de una vacuna
GET	`/mascotas/{mascota_id}/vacunaciones`	Historial de vacunas de una mascota
GET	`/mascotas/{mascota_id}/vacunaciones/seguimiento`	Seguimiento de las vacunas de una mascota
GET	`/vacunas/proximas`	Vacunas vencidas o por vencer (para recordatorios)
GET	`/esquemas-vacunacion`	Esquemas de vacunación de referencia
POST	`/emergencias`	Registrar una emergencia (sin cita previa)
GET	`/emergencias`	Listar emergencias (filtros opcionales)
POST	`/emergencias/{emergencia_id}/iniciar-atencion`	Iniciar la atención (IDENTIFICADA → EN_ATENCION)
GET	`/emergencias/{emergencia_id}`	Consultar una emergencia
GET	`/mascotas/{mascota_id}/evaluaciones/atencion`	¿La mascota puede recibir atención?
GET	`/avisos-veterinario`	Avisos de emergencia enviados al

Método	Ruta	Resumen
		veterinario
POST	`/mascotas/{mascota_id}/notas-clinicas`	Registrar una nota clínica en el expediente
GET	`/mascotas/{mascota_id}/expediente`	Consultar el expediente de la mascota
GET	`/notas-clinicas/{nota_id}`	Consultar una nota clínica
POST	`/hospitalizaciones`	Ingresar una mascota a hospitalización
GET	`/hospitalizaciones`	Listar hospitalizaciones (filtros opcionales)
GET	`/hospitalizaciones/{hospitalizacion_id}`	Consultar una hospitalización
POST	`/hospitalizaciones/{hospitalizacion_id}/monitoreo`	Agregar una lectura a la hoja de monitoreo
POST	`/hospitalizaciones/{hospitalizacion_id}/alta`	Dar de alta (INGRESADA → ALTA)
POST	`/mascotas/{mascota_id}/recetas`	Emitir una receta
GET	`/recetas/{receta_id}`	Consultar una receta
POST	`/mascotas/{mascota_id}/ordenes-estudio`	Emitir una orden de estudio
GET	`/ordenes-estudio/{orden_id}`	Consultar una orden de estudio
POST	`/citas/{cita_id}/cobro`	Cobrar una cita realizada
GET	`/citas/{cita_id}/cobro`	Consultar el cobro de una cita
POST	`/citas/{cita_id}/documentacion`	Entregar la documentación al propietario
GET	`/reportes/operacion`	Reporte de operación de la clínica

5. Trazabilidad: análisis, arquitectura, código y pruebas

Cada necesidad o regla del BRD se vincula con sus historias de usuario, el componente de diseño y de código que la cumple y las pruebas que la verifican. La matriz completa, con los nombres exactos de las pruebas, está en `docs/architecture/trazabilidad.md`; aquí se presenta una versión condensada.

Tabla 9. Matriz de trazabilidad condensada.

BRD	US	Componentes	Pruebas (archivos)	Estado
NB-01, NB-02, NB-03, R-10, S-01	US01, US02, US06	ServicioDeRegistro, Propietario, Mascota, repositorios_sql	test_servicios_de_aplicacion.py, test_dominio.py, test_repositorios_sql.py	Implementada
NB-04, NB-05, R-07	US08 (peso), US16 (alergias)	Atributos peso_kg y alergias de Mascota	test_repositorios_sql.py, test_dominio.py	Parcial (atributos; sin flujo clínico propio)
NB-06, R-05	US04, US07	ServicioDeCitas, Cita, ServicioSolicitado, reglas_cita (PuedeProcesarse)	test_reglas_cita.py, test_servicios_de_aplicacion.py, test_api.py, test_u5_matriz.py	Implementada
NB-08 (confirmar, modificar, cancelar)	US04	CicloDeVidaDeCita, Disponibilidad, PUT /citas/{id}/servicios, POST /citas/{id}/cancelar	test_ciclo_de_vida.py, test_disponibilidad.py, test_api.py	Implementada
NB-07	US04	PeriodoDeCita (diseño)	—	Diferida: los datos no traen horarios ni duraciones (A-11)
NB-20, R-01, R-09, R-14	Transversal	control_de_acceso (Rol, Permiso), encabezado X-Rol	test_control_de_acceso.py, test_api.py, test_api_errores.py	Implementada
NB-14, R-11	US15	ServicioDeVacunacion, Vacunacion, repositorios_sql	test_servicios_vacunacion.py, test_vacunacion.py, test_repositorios_vacunacion	Implementada

BRD	US	Componentes	Pruebas (archivos)	Estado
			.py	
NB-15, R-12	US15	EsquemaDeVacunacion, reglas_vacunacion, seguimiento_vacunacion	test_reglas_vacunacion.py, test_seguimiento_vacunacion.py, test_servicios_vacunacion.py	Implementada
NB-16, R-13	US15	seguimiento_vacunacion (es_refuerzo)	test_seguimiento_vacunacion.py, test_reglas_vacunacion.py	Implementada
NB-17	US05 (vacunas)	GET /vacunas/proximas	test_servicios_vacunacion.py, test_api.py	Implementada (cálculo por consulta; sin envío, A-05)
NB-19, R-06, S-08, CR-8	US20	ServicioDeEmergencias.atender_emergencia, Emergencia, CicloDeVidaDeEmergencia	test_servicios_emergencias.py, test_emergencia.py, test_api_emergencias.py	Implementada
NB-19 (notificar al veterinario)	US20	Puerto NotificadorDeVeterinario, BandejaDeAvisos	test_servicios_emergencias.py	Implementada
R-06 (atención permitida)	US20	atencion_permitida = TieneCitaConfirmada OR TieneEmergenciaActiva …	test_specification.py, test_servicios_emergencias.py	Implementada
Eventos de dominio (D-14)	Transversal	eventos, Despachador, tabla eventos_dominio	test_eventos.py, test_despachador.py, test_eventos_ciclo.py	Implementada (eventos de Cita, Vacunación, Emergencia, Nota clínica, Hospitalización, Receta, Orden de estudio,

BRD	US	Componentes	Pruebas (archivos)	Estado
				Cobro y Documentación)
Una transacción por caso de uso	Transversal	UnidadDeTrabajoSql	test_servicios_de_aplicacion.py, test_base_de_datos.py	Implementada
Contrato de errores HTTP (D-13)	Transversal	errores_http, main.py	test_api_errores.py, test_api.py	Implementada
Paquete del profesor: 60 operaciones, 120 casos, 20 transiciones	—	csv_loader, adaptador_paquete, sembrado, oráculo independiente	test_csv_loader.py, test_adaptador_paquete.py, test_oraculo.py, test_api_paquete.py, test_u5_matriz.py	Implementada
NB-10, NB-11, NB-12, R-02, R-03	US03, US11, US12	ServicioDeExpediente, NotaClinica, RegistroClinico, POST /mascotas/{id}/notas-clinicas …	test_expediente.py, test_api_clinica.py	Implementada (O-01, O-02, O-06)
NB-13, R-02	US13, US14	Receta, OrdenDeEstudio, EmisorDeDocumentos, POST /mascotas/{id}/recetas …	test_cobro_y_documentos.py, test_api_clinica.py	Implementada (O-10)
NB-18, R-04, R-08	US17, US18, US19	ServicioDeHospitalizacion, Hospitalizacion, CicloDeVidaDeHospitalizacion …	test_hospitalizacion.py, test_api_clinica.py	Implementada (O-03 a O-06)
NB-21	—	ServicioDeReportes, GET /reportes/operacion	test_cobro_y_documentos.py, test_api_clinica.py	Implementada (O-13)

BRD	US	Componentes	Pruebas (archivos)	Estado
Cobro y entrega de documentación (BPMN)	US10	Cobro, RegistroDeCobros, ServicioDeCobro, POST /citas/{id}/cobro …	test_cobro_y_documentos.py, test_api_clinica.py, test_money.py	Implementada (O-11, O-12; sin factura fiscal, A-02)

6. Pruebas, calidad y despliegue

6.1 Estrategia de pruebas

La verificación combina pruebas unitarias de dominio (reglas, Specifications, máquinas de estado, dinero), pruebas de aplicación (casos de uso con permisos, transacciones y eventos), pruebas de persistencia (repositorios SQL sobre SQLite y PostgreSQL 16) y pruebas de API por HTTP (códigos 200, 201, 400, 401, 403, 404, 409 y 422). Un oráculo independiente evalúa el texto de cada regla del paquete del profesor sobre las filas del CSV sin importar el dominio, y sus resultados se comparan con los de la API: 120 casos de prueba y 20 transiciones coinciden. Para CU-05, 20 escenarios por HTTP recorren todos los códigos del contrato y coinciden con el resultado derivado de los requerimientos, porque no existe oráculo oficial.

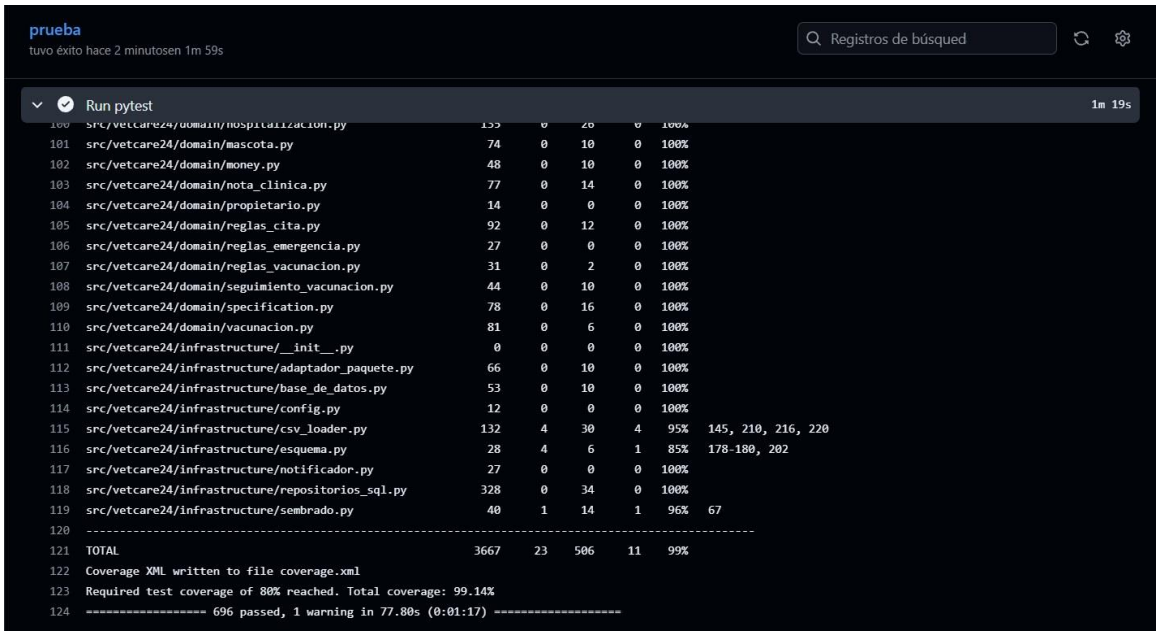


Figura 15. Integración continua: 696 pruebas aprobadas y 99.14 % de cobertura total (mínimo exigido: 80 %).

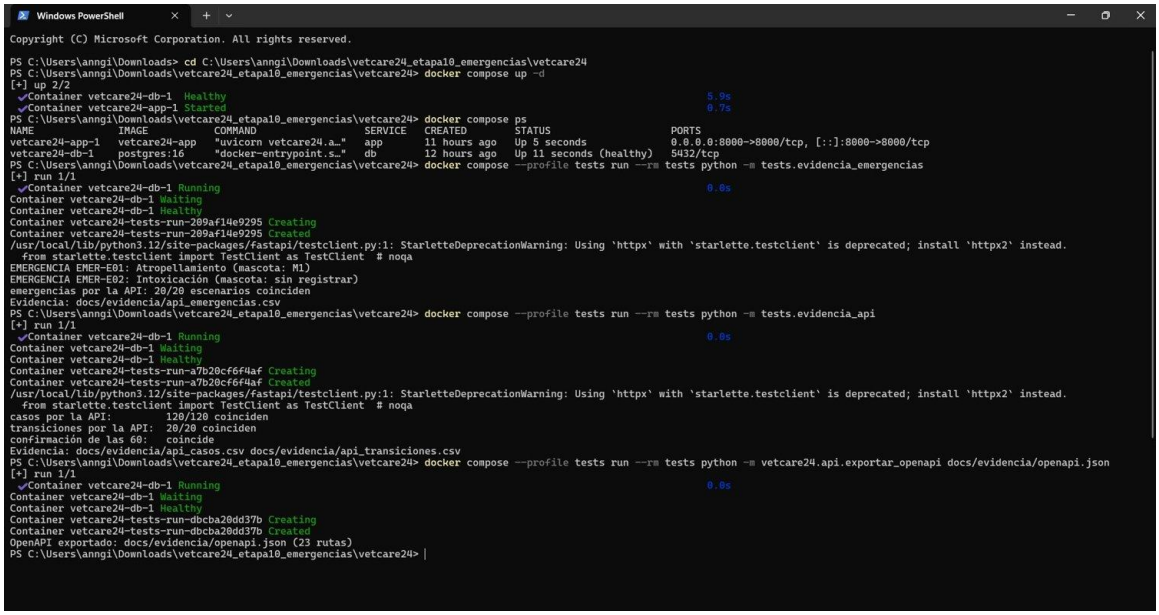


Figura 16. Evidencia por HTTP: 20 de 20 escenarios de emergencias, 120 de 120 casos y 20 de 20 transiciones.

6.2 CU-05 por HTTP en Swagger UI

Las capturas siguientes muestran el recorrido de CU-05 contra la API en ejecución: la evaluación de la regla R-06 antes de la emergencia, el alta de la emergencia (201), el rechazo cuando el rol no tiene permiso (403), el inicio de la atención por el personal clínico (200) y la segunda llamada, que responde 409 por transición inválida.

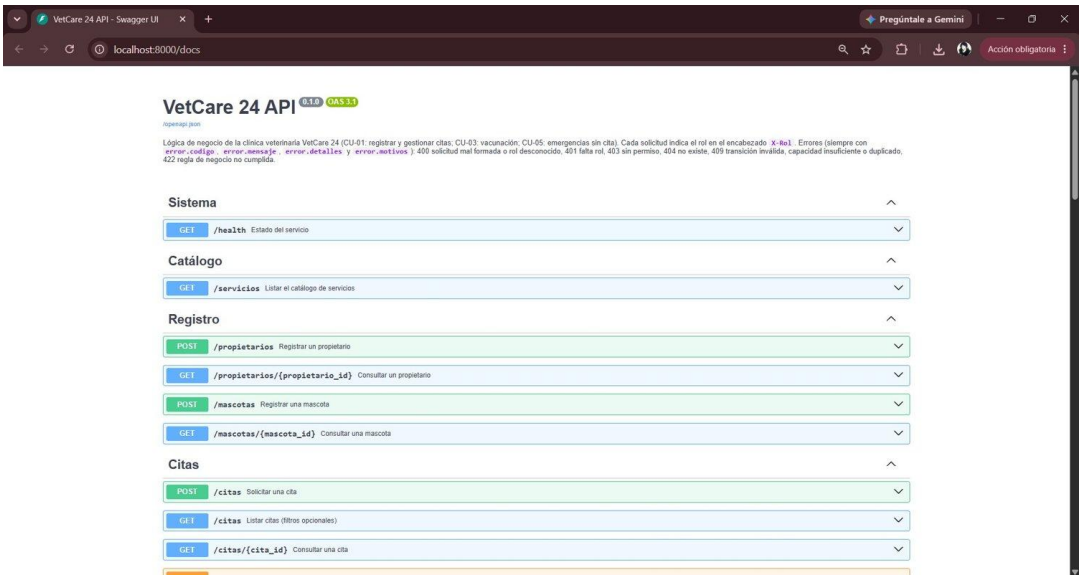


Figura 17. Swagger UI: secciones de la API.

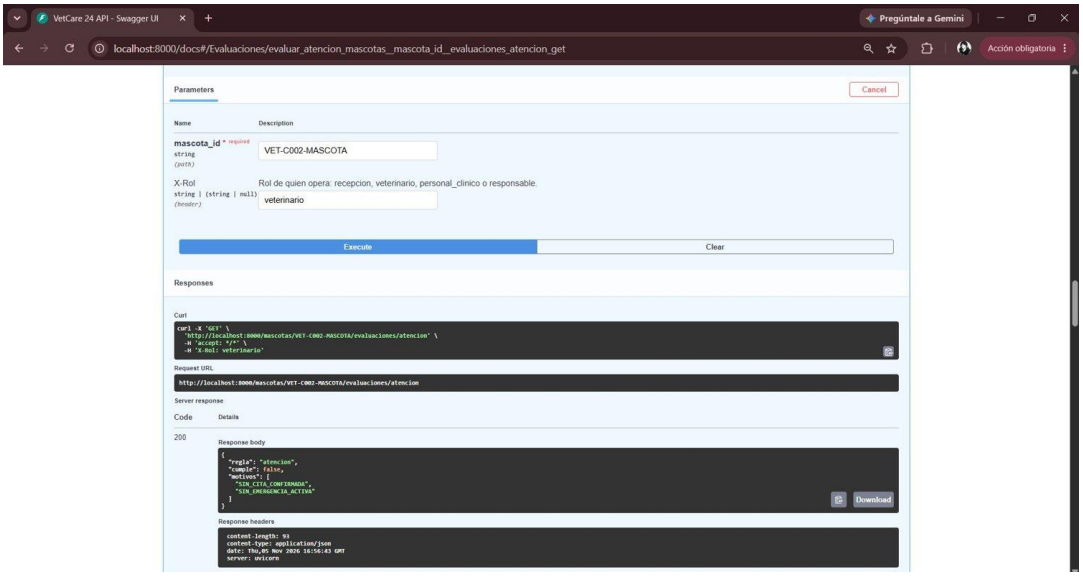


Figura 18. Evaluación de la regla de atención antes de registrar la emergencia.

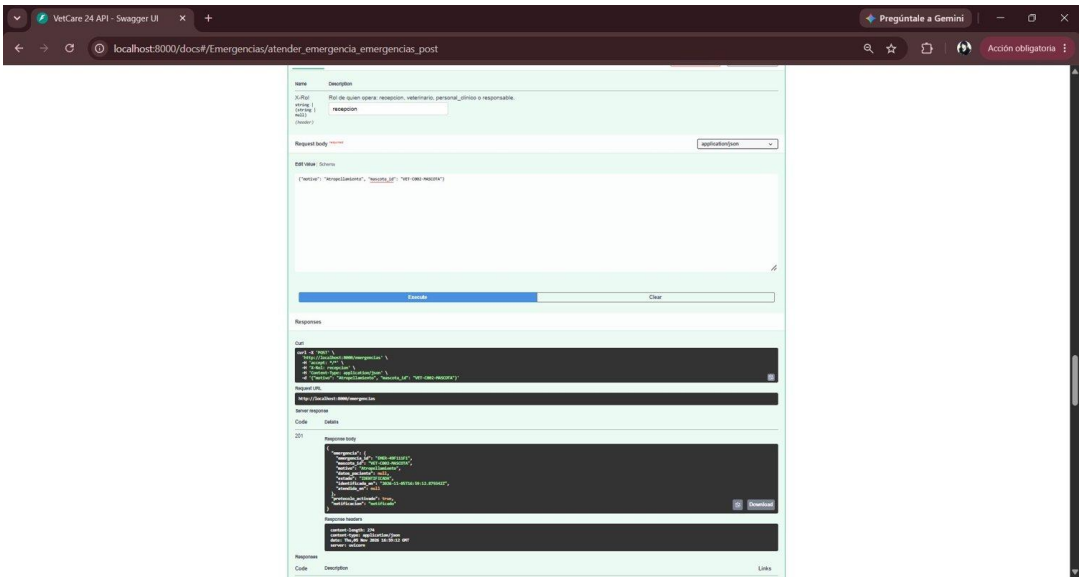


Figura 19. POST /emergencias: 201 con la emergencia IDENTIFICADA.

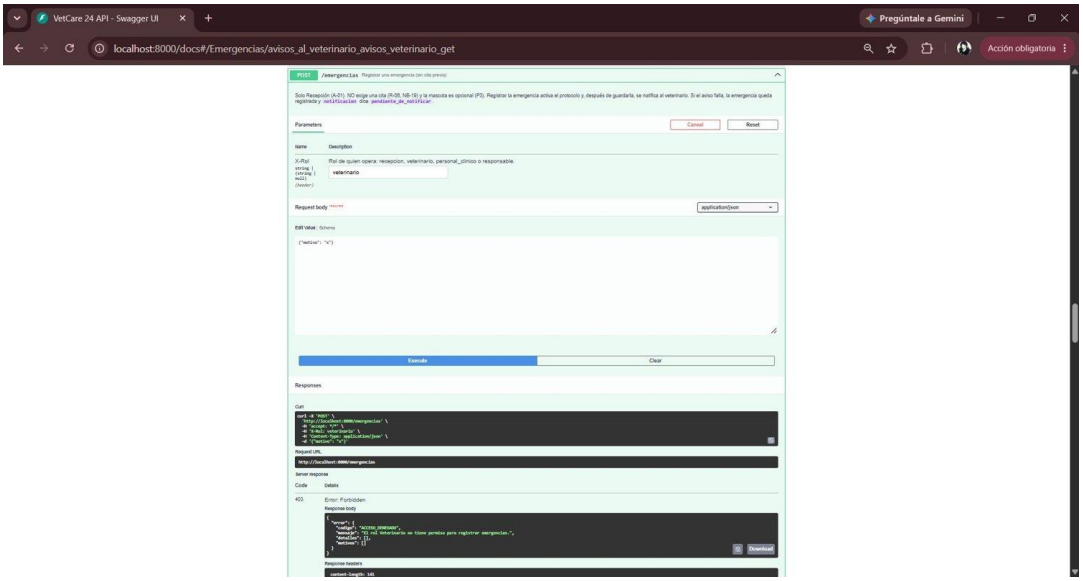


Figura 20. POST /emergencias con el rol de veterinario: 403.

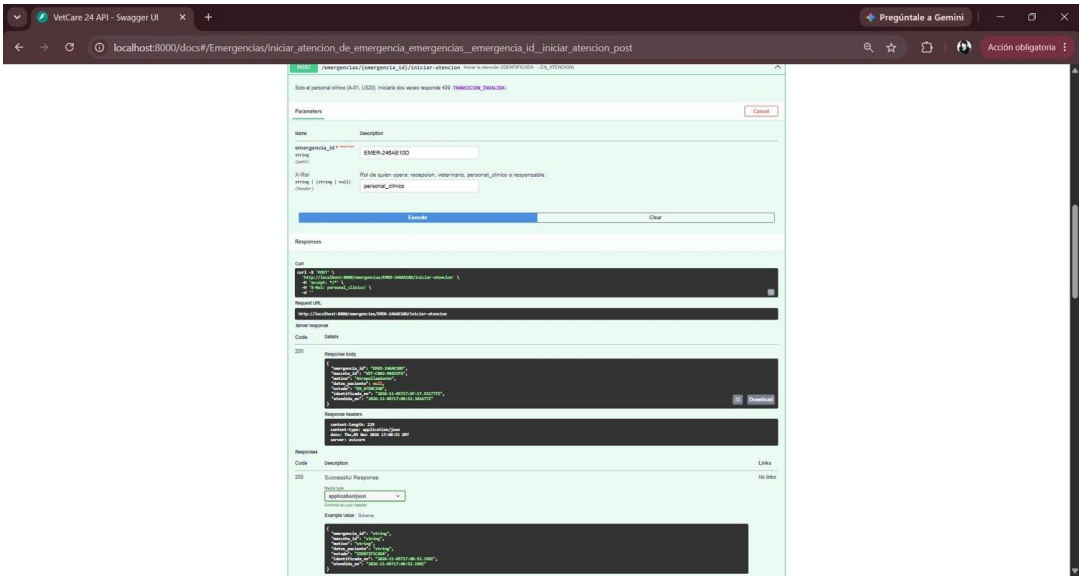


Figura 21. Inicio de la atención por el personal clínico: 200 EN_ATENCION.

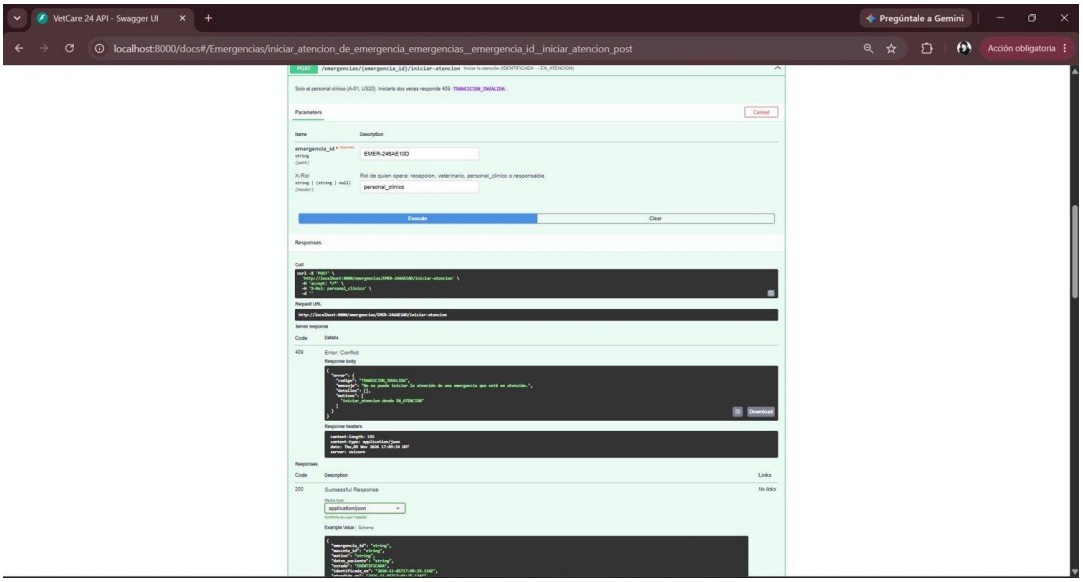


Figura 22. Segunda llamada de inicio de atención: 409 TRANSICION_INVALIDA.

6.3 Integración continua y control de calidad

El flujo de GitHub Actions ejecuta tres trabajos encadenados: `lint` (Ruff), `test` (pytest con PostgreSQL 16, cobertura con umbral de 80 % y generación de la evidencia y del contrato OpenAPI como artefactos) y `docker` (validación de Docker Compose y construcción de la imagen, con publicación en GHCR desde `main`). La rama `main` está protegida por un conjunto de reglas que exige pull request y que los controles pasen antes de integrar; un push directo se rechaza.

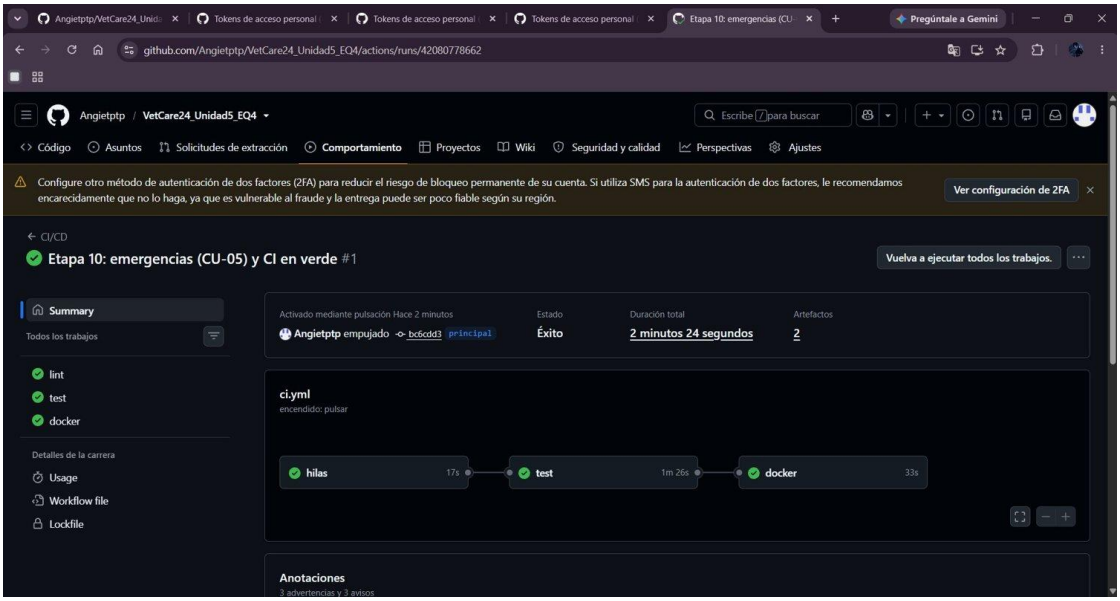


Figura 23. Pipeline de CI en verde: lint, test y docker.

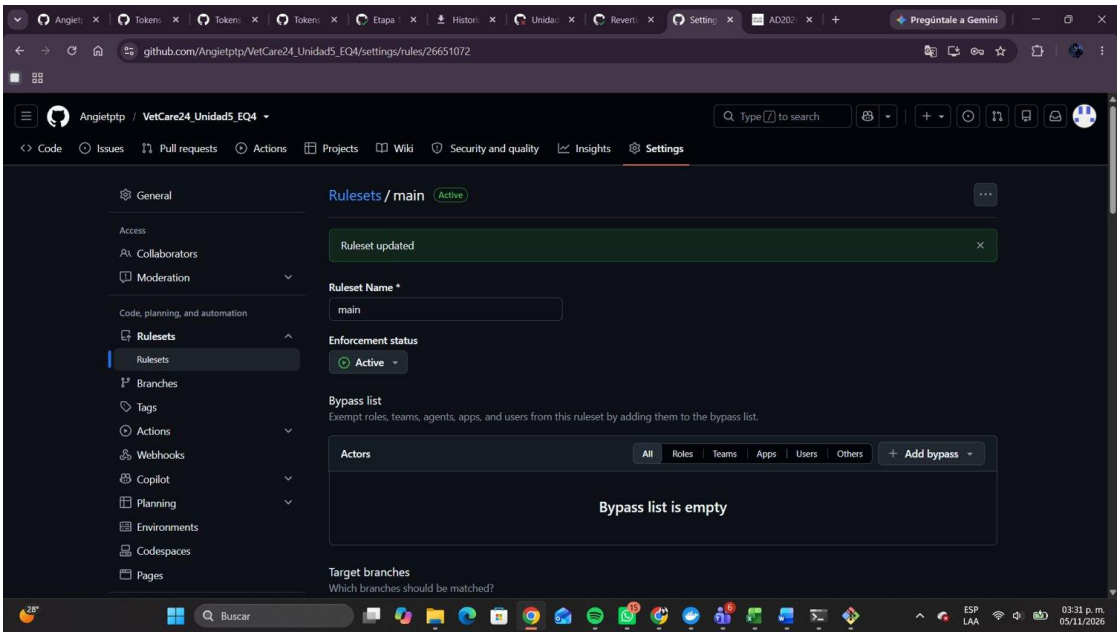


Figura 24. Conjunto de reglas activo en la rama principal.

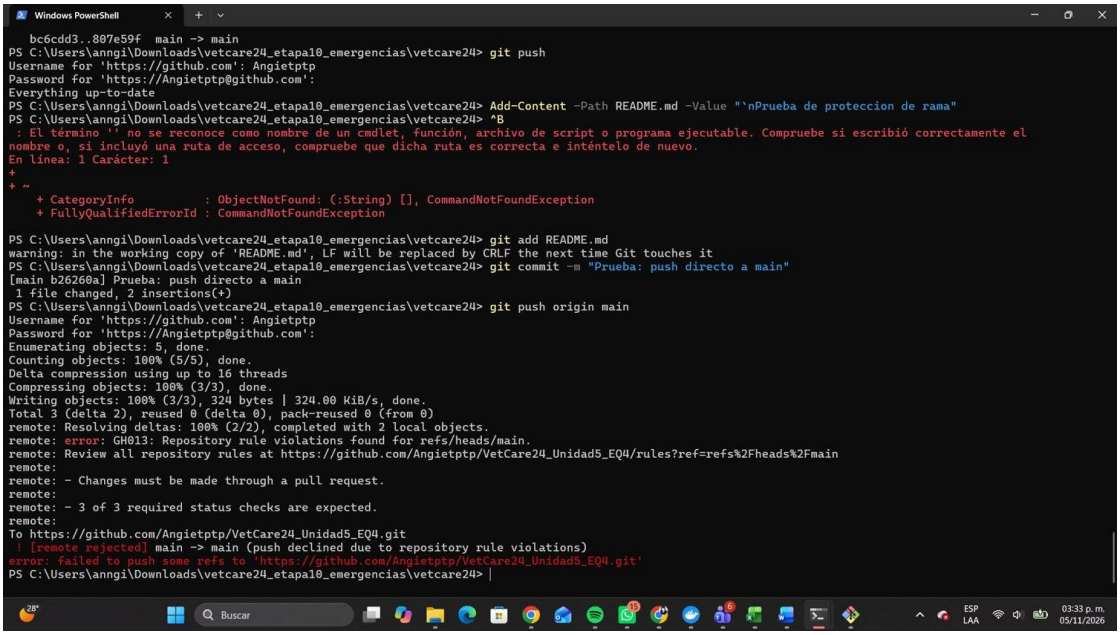


Figura 25. Push directo a main rechazado por la protección.

6.4 Contenedores y configuración

Docker Compose define tres servicios: `db` (PostgreSQL 16 con comprobación de salud), `app` (la API) y `tests` (perfil de pruebas con cobertura). La configuración sigue el enfoque de doce factores: `DATABASE_URL`, `DATA_DIR` y `PORT` se leen del entorno (`infrastructure/config.py`), `.env.example` documenta las variables y no se versionan secretos. El `Dockerfile` tiene las etapas `runtime`, `dev` y `prod`; la última, que es la que construyen las plataformas, arranca la API, siembra los datos si la base está vacía y escucha en `$PORT`. La aplicación acepta también URLs `postgres://`, que algunas plataformas entregan.

6.5 Despliegue

El servicio se desplegó como contenedor en Railway, con una base de datos administrada por la misma plataforma y la configuración por variables de entorno (`DATABASE_URL`, `PORT`). La verificación consistió en consultar `/health`, que responde `{"status":"ok"}`, y abrir Swagger UI en `/docs`. Durante el despliegue se resolvieron dos incidencias de configuración: la variable `DATABASE_URL` llegó vacía y la aplicación no arrancó, y el puerto público del dominio debió igualarse al puerto que la plataforma asigna al contenedor. Los pasos y las alternativas (Render, Fly.io) están en `docs/despliegue.md`.

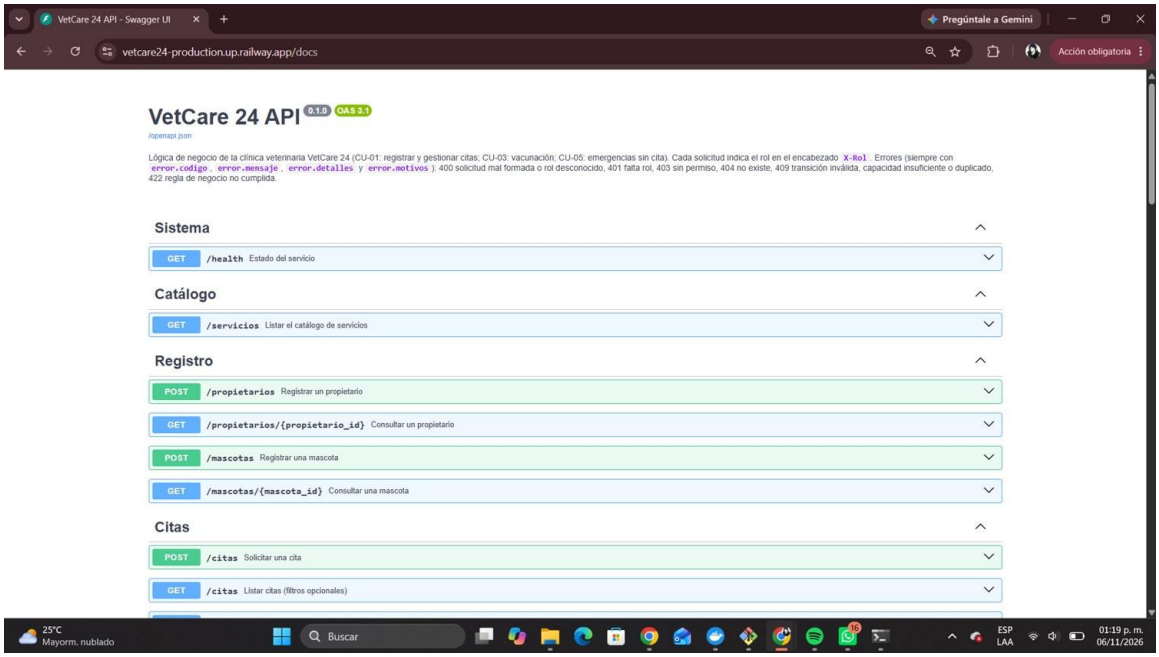


Figura 26. Swagger UI de la aplicación desplegada en Railway (vetcare24-production.up.railway.app/docs).

7. Limitaciones y trabajo futuro

El proyecto permitió recorrer el ciclo completo de una solución de lógica de negocio: desde el levantamiento de requerimientos y la redacción de historias de usuario con criterios verificables, hasta un diseño por capas con Domain-Driven Design, su implementación con pruebas automatizadas y su entrega mediante contenedores e integración continua. La trazabilidad entre necesidades del negocio, decisiones de arquitectura, código y pruebas facilita justificar cada regla ante el negocio y localizar el impacto de un cambio.

Entre las lecciones principales destacan tres. Primero, ubicar cada regla en el objeto que la protege y expresarla como Specification permitió explicar por qué una operación se rechaza, no solo que se rechaza. Segundo, aislar el paquete de pruebas del profesor detrás de un adaptador mantuvo el dominio limpio y permitió verificarlo con un oráculo independiente. Tercero, registrar las decisiones y sus correcciones, como el cambio de la fórmula de seguimiento de vacunas (M-05) o la ampliación del alcance a los cinco casos de uso (ADR-09), evitó que la documentación y el código divergieran.

Limitaciones

- **Alcance:** no hay factura fiscal (el cobro solo registra el pago, A-02), ni cobro por abonos, ni PDF de la documentación (el endpoint devuelve los datos), y el motivo de consulta (US09) se captura solo en emergencias.
- **Expediente:** las notas son inmutables y no hay versiones ni adjuntos; una corrección es una nota nueva. Los rangos de temperatura y frecuencia cardiaca de la hoja de monitoreo son una propuesta del equipo, sin validación clínica.
- **Autenticación:** el rol llega en un encabezado y no existe autenticación real.
- **Recordatorios:** se calculan por consulta y no se envían; el canal no está definido en el BRD. Los recordatorios de citas quedaron diferidos.
- **Fórmula de seguimiento de vacunas:** la próxima aplicación se calcula como última aplicación más la frecuencia del esquema; es una propuesta que debe validarse con el negocio, y los esquemas de prueba son ejemplos sin valor clínico.
- **Duración de las citas:** el modelo no incluye horarios ni duraciones porque los datos no los traen (NB-07 diferida).
- **Duplicados de mascotas:** el criterio para evitarlos (P-02, CR-2) sigue sin definirse.

Trabajo futuro

- Agregar adjuntos y resultados de laboratorio al expediente, y versiones de notas.
- Agregar autenticación y gestión de usuarios.
- Incorporar la factura fiscal, el PDF de la documentación y el envío real de recordatorios.
- Definir el criterio de detección de duplicados y el cierre de la emergencia.

Anexo A. Uso de herramientas de inteligencia artificial

El equipo utilizó un asistente de inteligencia artificial para generar borradores de código, pruebas, diagramas y documentación, y para revisar resultados. Cada entrega fue revisada, ejecutada y verificada por el equipo contra los documentos de las unidades anteriores. La bitácora de uso con las indicaciones, los resultados y las correcciones se conserva en el repositorio de la entrega de la unidad correspondiente.

8. Conclusiones

8.1 Conclusión general

El desarrollo de VetCare 24 permitió recorrer el ciclo completo de una solución de lógica de negocio, desde el análisis hasta su puesta en operación. Partimos de un problema real de la clínica: expedientes en papel, riesgo de registros duplicados y un seguimiento poco confiable de pacientes hospitalizados y de vacunación. A partir de ese análisis se redactaron las historias de usuario con criterios verificables, se modelaron los procesos en BPMN y se diseñó una arquitectura por capas con Domain-Driven Design. El resultado es una API desplegada y funcional que cubre de punta a punta los cinco casos de uso del diseño, así como el cobro, las recetas y órdenes de estudio, la entrega de documentación y el reporte de operación.

Las reglas del negocio quedaron protegidas en el dominio mediante agregados, máquinas de estado y eventos de dominio. Esto permite explicar por qué una operación se rechaza, no solo que se rechaza, y las respuestas de error se entregan al cliente en lenguaje claro y sin códigos técnicos. La calidad se respalda con 696 pruebas automatizadas y una cobertura de 99.14 % (el umbral exigido era 80 %), ejecutadas en una integración continua que valida el código, las pruebas con PostgreSQL y la imagen de Docker antes de cada despliegue en Railway. Las decisiones de arquitectura, sus correcciones y la trazabilidad entre necesidades, código y

pruebas quedaron documentadas, de modo que cualquier cambio futuro puede justificarse y su impacto localizarse con facilidad.

Como aprendizaje, el proyecto mostró que invertir tiempo en entender el negocio y en documentar las decisiones reduce los errores y el retrabajo en las etapas posteriores. También mostró que las pruebas automatizadas y la integración continua dan confianza para hacer cambios, como el de los mensajes de error o la ampliación del alcance a los cinco casos de uso, sin romper lo que ya funcionaba.

8.2 Conclusiones personales

- García Cerda Randal Guillermo:

El desarrollo de VetCare 24 me permitió comprender la importancia de analizar correctamente las necesidades de un negocio antes de comenzar a desarrollar una solución. A lo largo del proyecto, pude identificar cómo los problemas que presentaba la clínica, como el manejo de expedientes en papel, los registros duplicados y las dificultades para dar seguimiento a los pacientes, podían atenderse mediante un sistema organizado. También aprendí que las historias de usuario, los criterios de aceptación y los diagramas BPMN son herramientas importantes para establecer con claridad lo que se necesita desarrollar. Considero que uno de los aprendizajes más importantes fue reconocer que una buena solución no depende únicamente de que el sistema funcione, sino de que realmente responda a las necesidades de quienes lo utilizarán.

- Sarmiento Hernández Cesar Martín:

El desarrollo de VetCare 24 me permitió reconocer que la calidad de un sistema no se limita a implementar sus funcionalidades, sino que también requiere comprobar que estas trabajen correctamente y puedan mantenerse a largo plazo. A través de las pruebas automatizadas y la integración continua, comprendí cómo es posible detectar errores, verificar el comportamiento de la aplicación y realizar cambios con mayor confianza. Los resultados obtenidos, con 696 pruebas automatizadas y una cobertura del 99.14 %, demostraron la importancia de aplicar estos procesos durante el desarrollo. Además, aprendí que aspectos como la claridad de los mensajes de error, la documentación de las decisiones y la validación antes de cada despliegue también forman parte de una solución de calidad. En general, este proyecto me permitió relacionar los conocimientos de análisis, arquitectura, programación y pruebas en el desarrollo de un sistema funcional que responde a las necesidades planteadas.

- Torres Pasillas Angela Susana:

La realización de este proyecto me ayudó a comprender mejor la importancia de diseñar una arquitectura que permita organizar adecuadamente la lógica de negocio de un sistema. Mediante el uso de Domain-Driven Design, los agregados, las máquinas de estado y los eventos de dominio, pude identificar cómo se pueden proteger las reglas del negocio y mantener un funcionamiento consistente en los diferentes procesos de VetCare 24. Asimismo, comprendí que las decisiones tomadas durante el diseño deben estar bien justificadas y documentadas, ya que esto facilita entender el funcionamiento del sistema y realizar modificaciones posteriormente. Este trabajo me permitió reforzar mis conocimientos sobre análisis y diseño de software, además de reconocer la importancia de mantener una relación clara entre los requerimientos, la implementación y las pruebas.

Entregables:

Entregable	Ubicación
Repositorio de GitHub	https://github.com/Angietptp/VetCare24_PIA_EQ4
Despliegue funcional (Swagger UI)	https://vetcare24-production.up.railway.app/docs